

Índice

Lo primero que llama la atención	2
Gestión de Dependencias y Maven	3
Ignorando archivos de compilación (.gitignore)	3
Crear un nuevo proyecto desde cero	3
Sintaxis Básica	4
Llaves {} en vez de indentación	4
Punto y coma ; obligatorio	5
Variables y Tipos	5
Java es fuertemente tipado	5
Immutable vs Mutable	6
Visibilidad (public / private)	6
Por qué existe este concepto	6
¿Qué pasa si no ponemos modificador?	7
Niveles de visibilidad	7
Clases y Herencia	7
Herencia con extends	7
Clases abstractas	8
Clases concretas (no abstractas)	8
Métodos	9
return es obligatorio	9
void para métodos que no devuelven nada	9
Parámetros de métodos	10
Colecciones	10
Java necesita tipo específico y new	10
Recorrer una colección	10
Otros tipos de colecciones	10
Comparación Directa Wollok → Java	11
Override	12
Main - Punto de Entrada	12
¿Por qué tiene que ser exactamente ese método?	12
¿Cualquier clase puede tener el main?	13
¿Por qué existe Main.java al principio?	13
¿Cuándo se puede eliminar?	13
Ejemplo de main que hace algo	13
¿Qué pasa si el main está vacío?	14
¿Qué pasa si no tenemos main?	14
Constructores	14
Constructor en Java vs Wollok	14
¿Por qué this?	15
¿Qué pasa si no escribimos constructor?	15
Constructor en clase abstracta	15
¿Por qué protected en vez de private?	16

¿Qué pasa si no escribimos constructor y lo necesitamos?	16
Reporte de Cobertura con JaCoCo y Servidor Local	16
¿Qué es JaCoCo?	16
El problema al abrir el reporte directamente	16
La solución: Servidor local con Jetty	17
Cómo usarlo	17

- 1 Esta guía explica los conceptos básicos de Java comparándolos con WolloK, para
- 2 facilitar la traducción de un lenguaje a otro.

3 Lo primero que llama la atención

- 4 Lo primero que llama la atención es la estructura de directorios.

```

5
6 wtoj-plataforma-contenidos-CreadorSW-1
7   |
8   |— mvnw
9   |— mvnw.cmd
10  |— pom.xml
11  |— README.md
12  |— src
13     |— main
14        |— java
15           |— ar
16              |— edu
17                 |— unahur
18                    |— obj2
19                       |— w2j
20                          |— contenidos
21                             |— Contenido.java
22                             |— Peliculas.java
23                             |— series
24                                |— Episodio.java
25                                |— Serie.java
26                                |— Temporada.java
27                               |— Main.java
28  |— test
29     |— java

```

- 29 • Esa estructura es porque seguimos la convención de Maven.
- 30 • Cada clase en Java va en un archivo `.java` que se escribe con mayúscula.
- 31 • El `Main.java` viene con el repo como punto de entrada de ejemplo, pero no es
- 32 obligatorio para Maven. Esto se explica en detalle en la sección correspondiente.
- 33 • `mvnw` y `mvnw.cmd`: Scripts que permiten usar Maven sin instalarlo globalmente. El
- 34 primero es para Linux/Mac, el segundo para Windows.
- 35 • `pom.xml`: Archivo de configuración de Maven que define las dependencias y cómo
- 36 compilar el proyecto.

Gestión de Dependencias y Maven

Java utiliza herramientas como **Maven** para gestionar las librerías externas. Los editores modernos (como Zed) detectan automáticamente la configuración del proyecto:

- **Detección:** El servidor de lenguaje (JDT.LS) busca el archivo `pom.xml`. Si existe, configura el proyecto siguiendo el estándar de Maven.
- **Maven Wrapper (mvnw):** Permite ejecutar Maven sin tenerlo instalado globalmente, asegurando que todos los desarrolladores usen la misma versión.
- **Repositorio Local:** Las dependencias se descargan una sola vez a la carpeta `~/.m2/repository`.
 - La primera compilación es lenta porque descarga todo de internet.
 - Las siguientes son rápidas porque Maven reutiliza lo que ya tiene en disco.
 - Esto permite trabajar offline una vez que las dependencias iniciales fueron obtenidas.

Ignorando archivos de compilación (.gitignore)

Cuando subimos el proyecto a Git, **no** deben incluirse los archivos generados automáticamente:

- `target/`: Directorio donde Maven guarda los `.class` compilados y los resultados de los tests. Puede ocupar varios megabytes.
- `*.class`: Archivos de bytecode Java, por si compilamos un archivo suelto a mano fuera de Maven.

Estos archivos **no** deben subirse porque:

- Son generados automáticamente desde cualquier código fuente.
- Ocupan espacio innecesario.
- Cada desarrollador los genera localmente al compilar.

Todo lo demás **debe quedarse** en el repositorio:

- `.mvn/`, `mvnw`, `mvnw.cmd`, `pom.xml`, `src/`

Esto se configura en un archivo `.gitignore` en la raíz del proyecto.

Crear un nuevo proyecto desde cero

Para no tener que crear la estructura de directorios a mano, Maven tiene un **archetype** que la genera automáticamente:

```
1 mvn archetype:generate \  
2   -DgroupId=ar.edu.unahur.obj2 \  
3   -DartifactId=nombre-del-proyecto \  
4   -DarchetypeArtifactId=maven-archetype-quickstart \  
5   -DarchetypeVersion=1.5 \  
6   -DinteractiveMode=false
```



74 Esto crea:

```
75 1 nombre-del-proyecto/  
76 2 |─ pom.xml  
77 3 |─ src/  
78 4   |─ main/java/ar/edu/unahur/obj2/App.java  
79 5   |─ test/java/ar/edu/unahur/obj2/AppTest.java
```

80 Después solo hay que ajustar el `pom.xml` a la versión de Java y las dependencias
81 que se necesiten (JUnit 5, JaCoCo, etc.).

82 Sintaxis Básica

83 Llaves {} en vez de indentación

84 En Java, las llaves {} definen los bloques de código (clases, métodos, condiciones).
85 La indentación es solo visual, no sintáctica.

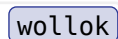
86 **Java:**

```
87 1 public class Contenido {  
88 2     private String titulo;  
89 3     private int costoBase;  
90 4  
91 5     public int costo() {  
92 6         return this.costoBase;  
93 7     }  
94 8 }
```



95 **Wollok:**

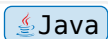
```
96 1 class Contenido {  
97 2     const titulo  
98 3     var costoBase  
99 4  
100 5     method costo() = costoBase  
101 6 }
```



102 ¿Qué pasa si no usamos las llaves?

103 **Java:**

```
104 1 public class Contenido {  
105 2     private String titulo // ERROR: missing } at end of class
```



106 El compilador tira error de sintaxis porque no puede determinar dónde termina el
107 bloque.

Punto y coma ; obligatorio

Java requiere ; al final de cada instrucción. Es obligatorio porque una línea puede contener varias instrucciones.

Java:

```
1 int x = 5;
2 x = x + 1;
```

 Java

Wollok:

```
1 var x = 5
2 x = x + 1
```

 wollok

¿Qué pasa si no ponemos ;?

Java:

```
1 int x = 5 // ERROR: syntax error, insert ';' to complete
  statement
2 x = x + 1;
```

 Java

El compilador informa exactamente dónde falta el ;.

Variables y Tipos

Java es fuertemente tipado

En Java **siempre** debemos indicar el tipo de la variable. No podemos dejar que el compilador lo infiera.

Java:

```
1 String titulo = "Recuerdos";
2 int costoBase = 20;
3 boolean esPopular = true;
4 double promedio = 22.5;
```

 Java

Wollok:

```
1 const titulo = "Recuerdos"
2 var costoBase = 20
3 var esPopular = true
4 var promedio = 22.5
```

 wollok

¿Qué pasa si usamos un tipo incorrecto?

Java:

```
139 1 String titulo = 123; // ERROR: incompatible types: int cannot
140   be converted to String
```

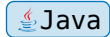


141 El compilador detecta el error antes de ejecutar el programa.

142 Immutable vs Mutable

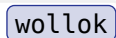
143 **Java:**

```
144 1 final String titulo = "Recuerdos"; // no se puede cambiar
145   (equivalente a const)
146 2 int costoBase = 20; // sí se puede cambiar
147   (equivalente a var)
```



148 **Wollok:**

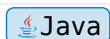
```
149 1 const titulo = "Recuerdos" // immutable
150 2 var costoBase = 20 // mutable
```



151 **¿Qué pasa si intentamos modificar una variable final?**

152 **Java:**

```
153 1 final String titulo = "Recuerdos";
154 2 titulo = "Otro título"; // ERROR: cannot assign a value to final
155   variable titulo
```



156 El compilador lanza error de compilación.

157 Visibilidad (public / private)

158 **Por qué existe este concepto**

159 El **encapsulamiento** protege los datos de modificaciones accidentales o
160 incorrectas. Cada clase decide qué propiedades y métodos son accesibles desde
161 afuera.

162 **Java:**

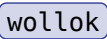
```
163 1 public class Contenido {
164 2     private String titulo; // solo la clase Contenido lo ve
165 3
166 4     public String getTitulo() { // acceso controlado
167 5         return this.titulo;
168 6     }
169 7
170 8     public void setTitulo(String nuevoTitulo) { // validación antes
171 9         de modificar
172         this.titulo = nuevoTitulo;
```



```
173 10    }
174 11 }
```

175 **Wollok:**

```
176 1 class Contenido {
177 2     const titulo
178 3     var costoBase
179 4 }
```



180 En Wollok no existe encapsulamiento: cualquiera puede ver y modificar cualquier
181 atributo.

182 **¿Qué pasa si no ponemos modificador?**

183 **Java:**

```
184 1 class Contenido {
185 2     String titulo; // package-private (default)
186 3 }
```



187 Cualquier clase en el **mismo paquete** puede ver y modificar `titulo`. Es una
188 vulnerabilidad porque se rompe el control sobre cómo se accede a los datos.

189 **Niveles de visibilidad**

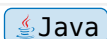
Modificador	Alcance
public	Cualquier clase desde cualquier paquete
private	Solo dentro de la misma clase
(ninguno)	Package-private (solo clases del mismo paquete)

194 **Clases y Herencia**

195 **Herencia con `extends`**

196 **Java:**

```
197 1 public class Pelicula extends Contenido {
198 2     // hereda titulo y costoBase automáticamente
199 3 }
```



200 **Wollok:**

```
201 1 class Pelicula inherits Contenido {  
202 2     // vacía, hereda todo  
203 3 }
```

wollok

204 **¿Qué pasa si heredamos de una clase que no existe?**

205 **Java:**

```
206 1 public class Pelicula extends Contenido { }  
207 2 // ERROR: Contenido cannot be resolved to a type
```

 Java

208 El compilador no encuentra la clase padre.

209 Clases abstractas

210 Una clase abstracta no se puede instanciar directamente. Sirve como base para
211 otras clases.

212 **Java:**

```
213 1 public abstract class Contenido {  
214 2     public abstract int costo(); // las subclasses DEBEN implementar  
215 3 }
```

 Java

216 **Wollok:**

```
217 1 class Contenido {  
218 2     method costo() // sin cuerpo = abstracto  
219 3 }
```

wollok

220 **¿Qué pasa si no implementamos el método abstracto?**

221 **Java:**

```
222 1 public class Pelicula extends Contenido {  
223 2     // ERROR: The type Pelicula must implement the inherited abstract  
224 2     method Contenido.costo()  
225 3 }
```

 Java

226 El compilador obliga a implementar todos los métodos abstractos.

227 Clases concretas (no abstractas)

228 **Java:**

```
229 1 public class Pelicula extends Contenido {  
230 2     @Override  
231 3     public int costo() {  
232 4         return this.getCostoBase();  
233 5     }
```

 Java

234 6 }

235 **Wollok:**

```
236    1 class Pelicula inherits Contenido {  
237    2     // vacía, hereda el costo() de Contenido si existiera  
238    3 }
```

wollok

239 Métodos

240 **return es obligatorio**

241 Si declaramos un método que devuelve algo, **debemos** retornar un valor.

242 **Java:**

```
243    1 public int costo() {  
244    2     return this.costoBase;  
245    3 }
```

Java

246 **Wollok:**

```
247    1 method costo() = costoBase
```

wollok

248 **¿Qué pasa si no ponemos return?**

249 **Java:**

```
250    1 public int costo() {  
251    2     costoBase; // ERROR: missing return statement  
252    3 }
```

Java

253 El compilador detecta que el método promete devolver int pero no lo hace.

254 **void para métodos que no devuelven nada**

255 **Java:**

```
256    1 public void agregarTemporada(Temporada t) {  
257    2     temporadas.add(t);  
258    3 }
```

Java

259 **Wollok:**

```
260    1 method agregarTemporada(unaTemporada) {  
261    2     temporadas.add(unaTemporada)  
262    3 }
```

wollok

263 **¿Qué pasa si intentamos usar return en un método void?**

264 **Java:**

```
265 1 public void agregarTemporada(Temporada t) {  
266 2     return true; // ERROR: void methods cannot return a value  
267 3 }
```

 Java

268 Parámetros de métodos

269 **Java:**

```
270 1 public void agregarTemporada(Temporada unaTemporada) {  
271 2     temporadas.add(unaTemporada);  
272 3 }
```

 Java

273 **Wollok:**

```
274 1 method agregarTemporada(unaTemporada) {  
275 2     temporadas.add(unaTemporada)  
276 3 }
```

 wollok

277 Colecciones

278 **Java necesita tipo específico y new**

279 **Java:**

```
280 1 List<Temporada> temporadas = new ArrayList<>();  
281 2 temporadas.add(unaTemporada);  
282 3 temporadas.size();
```

 Java

283 **Wollok:**

```
284 1 const temporadas = []  
285 2 temporadas.add(unaTemporada)  
286 3 temporadas.size()
```

 wollok

287 Recorrer una colección

288 **Java:**

```
289 1 int suma = temporadas.stream().mapToInt(t -> t.costo()).sum();
```

 Java

290 **Wollok:**

```
291 1 var suma = temporadas.sum({ t => t.costo() })
```

 wollok

292 Otros tipos de colecciones

293 **Java:**

```

294 1 Set<String> titulos = new HashSet<>();
295 2 Map<String, Integer> duracion = new HashMap<>();

```

Java

296 **Wollok:**

```

297 1 // Wollok no distingue entre Set, Map, List
298 2 // usa simplemente listas
299 3 const titulos = []
300 4 const duracion = []

```

wollok

301 Comparación Directa Wollok → Java

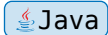
302 Concepto	Wollok	Java
303 Inmutable	const	final
304 Mutable	var	tipo (sin final)
305 Herencia	class X inherits Y	class X extends Y
306 Clase abstracta	método sin cuerpo	abstract class X
307 Method abstracto	method costo()	public abstract int costo()
308 Visibilidad	no existe	public / private
309 Lista	[]	new ArrayList<>()
310 Tamaño	.size()	.size()
311 Sumar elementos	.sum({x => x.costo()})	.stream().mapToInt(x -> x.costo()).sum()
312		
313 Punto y coma	opcional	obligatorio
314 Llaves	no se usan	obligatorias
315 Constructor	automático	hay que escribirlo

Override

En Java es **obligatorio** usar la anotación `@Override` cuando sobrescribimos un método de la clase padre.

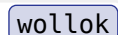
Java:

```
1 @Override
2 public int costo() {
3     return 3;
4 }
```



Wollok:

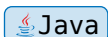
```
1 override method costo() {
2     return 3
3 }
```



¿Qué pasa si nos olvidamos de `@Override`?

Java:

```
1 public int costo() { // se supone que es override pero sin
331 anotación
2     return 3;
3 }
```



Si no es realmente un override (por ejemplo, si el método padre tiene otra firma), Java **no nos avisa**. Podemos tener un método completamente nuevo sin querer y el compilador no dice nada.

Main - Punto de Entrada

Todo programa Java necesita un método `main` como punto de entrada. La JVM busca específicamente `public static void main(String[] args)` para saber dónde empezar.

¿Por qué tiene que ser exactamente ese método?

La firma debe ser literalmente `public static void main(String[] args)` porque es el contrato que la JVM espera:

- **public:** la JVM necesita poder acceder al método desde fuera de la clase.
- **static:** se ejecuta sin necesidad de crear una instancia de la clase.
- **void:** no devuelve nada a la JVM.
- **main:** es el nombre exacto que la JVM busca.
- **`String[] args`:** recibe los argumentos que se pasan por línea de comandos.

Si cambiamos algo (por ejemplo, `public static void start(String[] args)`), la JVM no lo reconoce y tira error.

¿Cualquier clase puede tener el main?

Sí, pero no literalmente cualquier clase: tiene que ser una clase que contenga el método exacto `public static void main(String[] args)`. El nombre de la clase puede ser cualquiera (`Main.java`, `App.java`, `Launcher.java`, etc.). Maven y Java no obligan a que se llame `Main`, ese nombre es solo una convención.

Poner el `main` en una clase de dominio (como `Contenido.java` o `Plataforma.java`) funciona, pero no es una buena práctica porque mezcla la infraestructura (punto de entrada) con la lógica de negocio. Es mejor mantenerlo en una clase separada.

¿Por qué existe `Main.java` al principio?

El `Main.java` que viene en el repositorio inicial es una decisión de los docentes, no un requisito de Maven. Maven compila el proyecto y genera la estructura de directorios basándose únicamente en el archivo `pom.xml` y en la convención de directorios (`src/main/java`, `src/test/java`, etc.). No necesita un archivo `Main.java` para funcionar.

¿Cuándo se puede eliminar?

Si nuestro proyecto solo contiene clases de dominio y tests con JUnit, podemos eliminar el `Main.java` sin problemas. La ejecución de los tests se realiza mediante `mvn test` a través del plugin Surefire, que no requiere un método `main`.

Es importante entender que al eliminar el `Main.java`, nuestro proyecto como programa ejecutable no está haciendo nada. Los tests verifican que las clases de dominio funcionen correctamente, pero no ejecutan el sistema: instancian objetos, les mandan mensajes y comprueban resultados, pero no ponen en marcha una aplicación. Si queremos que nuestro proyecto haga algo como programa (por ejemplo, mostrar información en consola o iniciar una interfaz), necesitamos un `main` que cree objetos y coordine el comportamiento. Solo necesitamos un `main` si vamos a ejecutar nuestro programa como una aplicación standalone (por ejemplo, con `java -jar` o `mvn exec:java`).

Ejemplo de `main` que hace algo

El `main` nunca debe tener toda la lógica. Solo crea objetos y delega.

```
1 package ar.edu.unahur.obj2.w2j;
2
3 import ar.edu.unahur.obj2.w2j.contenidos.Pelicula;
4 import ar.edu.unahur.obj2.w2j.planes.PlanBasico;
5 import ar.edu.unahur.obj2.w2j.planes.Usuario;
6
7 public class Main {
8     public static void main(String[] args) {
9         // Crear contenidos
```

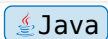


```
389 10      Pelicula pelicula1 = new Pelicula("Recuerdos", 20.0);
390 11      Pelicula pelicula2 = new Pelicula("Avatar", 30.0);
391 12
392 13      // Crear un usuario con plan básico
393 14      PlanBasico plan = new PlanBasico(100.0);
394 15      Usuario usuario = new Usuario("Juan", plan);
395 16
396 17      // El usuario mira contenidos
397 18      usuario.verContenido(pelicula1);
398 19      usuario.verContenido(pelicula2);
399 20
400 21      // Mostrar resultados
401 22      System.out.println("Usuario: " + usuario.getNombre());
402 23      System.out.println("Costo total del plan: " +
403      usuario.getPlan().getCosto());
404 24  }
405 25 }
```

406 ¿Qué pasa si el main está vacío?

407 El programa compila y corre, pero no hace nada visible. Las clases de dominio
408 existen en el código, pero si nadie las instancia dentro del main, simplemente
409 «están ahí».

```
410 1 public static void main(String[] args) {
411 2     // vacío
412 3 }
```



413 ¿Qué pasa si no tenemos main?

414 Si intentamos ejecutar el programa con java o mvn exec:java pero ninguna clase
415 tiene el método main, obtenemos:

```
416 1 Error: Main method not found in class Contenido, please define the main
417 1 method as:
418 2 public static void main(String[] args)
```

419 Constructores

420 En Java, si necesitamos inicializar objetos con parámetros, debemos escribir el
421 constructor explícitamente. Wollok lo hace automáticamente, Java no.

422 Constructor en Java vs Wollok

423 **Java:**

```
424 1 public class Contenido {
425 2     private final String titulo;
426 3     private double costoBase;
427 4
428 5     public Contenido(String titulo, double costoBase) {
429 6         this.titulo = titulo;
430 7         this.costoBase = costoBase;
431 8     }
432 9 }
```

 Java

433 **Wollok:**

```
434 1 class Contenido {
435 2     const titulo
436 3     var costoBase
437 4 }
438 5 // Wollok genera automáticamente el constructor con los parámetros
```

 wollok

439 **¿Por qué this?**

440 this.titulo significa «el campo titulo de este objeto». Sin this, Java no sabe si
441 titulo se refiere al parámetro o al campo.

```
442 1 public Contenido(String titulo, double costoBase) {
443 2     this.titulo = titulo;           // this.titulo = campo, titulo =
444 3     parámetro
445 4     this.costoBase = costoBase;
446 5 }
```

 Java

447 **¿Qué pasa si no escribimos constructor?**

```
448 1 public class Contenido {
449 2     private String titulo;
450 3     private double costoBase;
451 4 }
452 5 // Java crea automáticamente: Contenido() {}
```

 Java

453 Si no definimos ningún constructor, Java crea uno automático sin parámetros. Pero
454 si definimos **al menos uno con parámetros**, el automático desaparece.

455 **Constructor en clase abstracta**

```
456 1 public abstract class Contenido {
457 2     private final String titulo;
458 3     protected double costoBase; // protected para que las subclases
459 4     accedan
```

 Java

```

460 4
461 5     public Contenido(String titulo, double costoBase) {
462 6         this.titulo = titulo;
463 7         this.costoBase = costoBase;
464 8     }
465 9 }

```

466 **¿Por qué `protected` en vez de `private`?**

Modificador	Quién puede verlo
<code>private</code>	Solo la clase misma
<code>protected</code>	La clase y sus subclases

470 Si `costoBase` fuera `private`, `Pelicula` (que hereda de `Contenido`) no podría acceder
471 a `costoBase`.

472 **Wollok:** No existe este concepto. Todos los atributos son accesibles.

473 **¿Qué pasa si no escribimos constructor y lo necesitamos?**

```

474 1  Contenido c = new Contenido(); // ERROR: no default constructor
475 exists

```

476 Si definimos un constructor con parámetros y necesitamos el sin parámetros,
477 debemos escribirlo nosotros.

478 Reporte de Cobertura con JaCoCo y Servidor 479 Local

480 **¿Qué es JaCoCo?**

481 JaCoCo es una herramienta que genera reportes de cobertura de código, es decir,
482 indica qué partes de nuestro código fueron ejecutadas por los tests y cuáles no. El
483 reporte se genera como una página HTML dentro de la carpeta `target/site/`
484 `jacoco/`.

485 **El problema al abrir el reporte directamente**

486 Al abrir el archivo `index.html` del reporte haciendo doble clic, el navegador lo
487 carga con el protocolo `file:///`. Algunos navegadores (como Chromium o
488 Ungogled Chromium) bloquean la carga de recursos locales (CSS y JavaScript)
489 por políticas de seguridad. Esto hace que el reporte se vea incompleto o sin estilos.
490 Firefox, en cambio, suele permitirlo.

La solución: Servidor local con Jetty

Para evitar este problema, podemos levantar un servidor web local que sirva el reporte mediante el protocolo `http://`. Esto se logra agregando el plugin de Jetty al archivo `pom.xml`:

```
1  <plugin>
2      <groupId>org.eclipse.jetty</groupId>
3      <artifactId>jetty-maven-plugin</artifactId>
4      <version>11.0.16</version>
5      <configuration>
6          <supportedPackagings>
7              <supportedPackaging>jar</supportedPackaging>
8          </supportedPackagings>
9          <webApp>
10             <resourceBase>${project.build.directory}/site/jacoco</
11             resourceBase>
12          </webApp>
13          <httpConnector>
14              <port>8080</port>
15          </httpConnector>
16      </configuration>
17  </plugin>
```

Cómo usarlo

Una vez que el reporte de JaCoCo fue generado (con `mvn test`), ejecutamos:

```
1  mvn jetty:run
```

Esto levanta un servidor local en `http://localhost:8080`. Al abrir esa dirección en cualquier navegador, el reporte se visualiza correctamente con todos sus estilos y funcionalidades.

La primera vez que se ejecuta, Maven descarga el plugin de Jetty y sus dependencias. Esto sucede una sola vez y se almacena en el repositorio local (`~/.m2/repository`). Las siguientes ejecuciones son instantáneas, incluso sin conexión a internet.